Nyelvek típusrendszere vizsga (B csoport)

2024. december 30.

Név és Neptun kód: _____

Ott van minden feladat mellett, hogy mennyi pontot ér. Alapértelmezetten minden feladat többszörös választás (nulla vagy több megoldást kell bekarikázni). Ha nem az, akkor oda van írva. Összesen 70 pont.

1 Programozási nyelv

A vizsgában az összes kérdés az alábbi programozási nyelvre vonatkozik. Ez a System F, szabályai az alábbiak (ugyanazok, mint előadáson).

```
Ty      : Set
Tm     : Ty → Set _⇒_      : Ty → Ty
→ Ty lam      : (Tm A → Tm B) → Tm
(A ⇒ B)
_·_      : Tm (A ⇒ B) → Tm A → Tm B
⇒β       : lam t · a = t a
⇒η       : (f : Tm (A ⇒ B)) → f = lam (λx.f·x)
∀        : (Ty → Ty) → Ty
Lam      : ((A : Ty) → Tm (F A)) → Tm (∀ F)
_·_      : Tm (∀ F) → (A : Ty) → Tm (F A)
∀β       : Lam t · A = t A
∀η       : (f : Tm (∀ F)) → f = Lam (λX.f·X)
```

2 Lexing, parsing

Az alábbi lexikális elemeket használjuk: `lam`, `·`, `Lam`, `·`, `.`, `(`, `)`, `X`, `Y`, `Z`, …, `x`, `y`, `z`, … A típusozatlan AST-k az alábbi módon vannak megadva:

```
TyVar := String
TmVar := String
```

```
Ty : Set
var : TyVar → Ty
_⇒_ : Ty → Ty → Ty

∀   : TyVar → Ty → Ty
```

```
Tm : Set
var : TmVar → Tm
lam : TmVar → Tm →
Tm
_·_ : Tm → Tm → Tm
Lam : TyVar → Tm →
Tm
_·_ : Tm → Ty → Tm
```

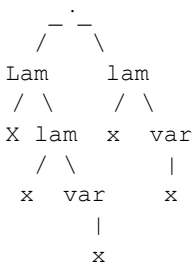
1. (1 pont) Tekintsük az alábbi sztringet:

   ```
   Lam X . Lam Y . (lam x . lam . x
   ```

   Ebből a sztringből egy lexer melyik lexikális elem-sorozatot adja végeredményül? (egyszeres választás)

   - [`Lam`, `X`, `.`, `Lam`, `Y`, `.`, `(`, `lam`, `x`, `.`, `lam`, `.`, `x`, `)`]
   - [`Lam`, `X`, `.`, `Lam`, `Y`, `.`, `(`, `lam`, `x`, `.`, `x`, `)`]
   - [`Lam`, `X`, `.`, `Lam`, `Y`, `.`, `(`, `lam`, `x`, `.`, `lam`, `.`, `x`]
   - [`Lam`, `X`, `.`, `Lam`, `Y`, `.`, `lam`, `x`, `.`, `lam`, `.`, `x`]
   - [`Lam`, `X`, `.`, `Lam`, `Y`, `.`, `lam`, `x`, `.`, `x`]
   - Egyiket sem, mert nincs jól zárójelezve a kifejezés.
   - Egyiket sem, mert a `lam` után mindig változónak kell szerepelnie, és a második `lam` után nincs változó.

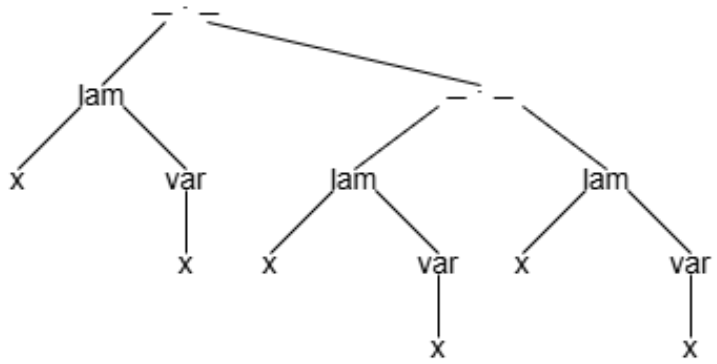
2. (1 pont) Az alábbi fát hogyan írjuk le típusozatlan AST-ként? (egyszeres választás)



- `(lam x . var x) · (lam x . var x)`
- `(Lam X . lam x . var x) · (lam x . var x)`
- `Lam X . ((lam x . var x) · lam x . var x)`
- `Lam X . lam x . lam x . var x · var x`
- Nem írható le Agdában, mert a `_ · _` első paramétere ⇒ típusú, a `Lam` viszont ∀ típusú.

3. (2 pont) Rajzold le, hogy az alábbi sztringből melyik absztrakt szintaxisfát hozza létre a parser (az előző feladat fordítottja)!

`(lam x . var x) · ((lam x . var x) · (lam x . var x))`



3 Típuskikövetkeztetés

Az implicit paramétereket kapcsos zárójelekkel jelöljük, például `lam {A}{B} : (Tm A → Tm B) → Tm (A ⇒ B)`, a következő esetben csak az első implicit paramétert adjuk meg: `lam {A} (λx.x) : Tm (A ⇒ A)`.

A típustörlés `Tm A` egy eleméből egy típusozatlan AST-t hoz létre. Például `Lam λX.Lam λY.lam {X}{Y⇒X} λx.lam {Y}{X} λy.x`-ból `Lam X . Lam Y . lam x . lam y . var x` lesz. A típuskikövetkeztetés a típustörlés ellentéte: egy típusozatlan AST-ből egy `A` típust és `Tm A` egy elemét hozza létre.

4. (2 pont) Jelöld be azokat az `A` típusokat, melyekhez létezik `t : Tm A`, hogy `t` típustörlése a `lam x . var x` típusozatlan AST-t eredményezi.

- `(∀ λX.X) ⇒ (∀ λX.X)`
- `(∀ λX.X ⇒ X) ⇒ (∀ λX.X)`
- `(∀ λX.X⇒X⇒X) ⇒ (∀ λX.X⇒X⇒X)`
- `(∀ λX.X⇒X) ⇒ (∀ λX.X⇒X)`
- `(∀ λX.X) ⇒ (∀ λX.X⇒X)`
- `(∀ λX.(X⇒X)⇒X) ⇒ (∀ λX.(X⇒X)⇒X)`

Indoklás: lényegében az identitásfüggvényről van szó, tehát ⇒ mindkét oldalára ugyanaz kell.

5. (2 pont) Az alábbiakban felsorolunk típusozatlan AST-ket. Egy típuskikövetkeztető ezek közül melyiket utasítja biztosan vissza? (Tehát mely AST-khez nem létezik olyan `A : Ty` és `t : Tm A`, hogy `t` típustörlése az AST-t adja vissza.)

   - `lam x . var x`
   - `lam x . lam y . var x · var y`
   - `lam x . lam y . var y · var y`
   - `lam x . lam y . var y · var x`
   - `lam x . lam y . var y · (var y · var x)`
   - `lam x . var x · var x`
   - `(lam x . var x) · (lam y . var y)`
   - Minden AST-hez létezik `A : Ty` és `t : Tm A`, hogy `t` típustörlése az AST-t adja vissza.

   Indoklás: Az önalkalmazás van visszautasítva, hiszen ez nem típusozható a rendszerben.

6. (6 pont) Az alábbiakban felsorolunk jól típusozott termeket a nyelvünkben. Írd melléjük a típusukat! Az elsőt megadtuk példaként.

```
Lam λX.lam {X} λx.x                                  : Tm (∀ λX. X ⇒ X)

Lam λX.lam {X ⇒ X} λf.lam {X} λx.f · (f · x)         : Tm (∀ λX.(X ⇒ X) ⇒ X ⇒ X)

lam {∀ λX.X} λx.x                                    : Tm (∀ λX. X ⇒ ∀ λX. X)

Lam λX.lam {X} λx.Lam λY.lam {Y} λy.y                : Tm (∀ λX. X ⇒ ∀ λY. Y ⇒ Y)

Lam λX.lam {X} λx.lam {X} λy.x                       : Tm (∀ λX. X ⇒ X ⇒ X)

Lam λX.lam {X ⇒ X} λf.lam {X ⇒ X} λg.f               : Tm (∀ λX.(X⇒X) ⇒ (X⇒X) ⇒ (X⇒X))

Lam λX.lam {X} λx.lam {X} λy.lam {X} λz.x            : Tm (∀ λX. X ⇒ X ⇒ X ⇒ X)
```

7. (5 pont) Írj tetszőleges adott típusú termet, ha van! Ha nincs, írd oda, hogy nincs! Az elsőt megadtuk példaként.

```
Lam λX.lam λx.x                                  : Tm (∀ λX.X ⇒ X)

Lam λX.lam λx.lam {X} λy.x                       : Tm (∀ λX.X ⇒ (X ⇒ X))

nincs                                            : Tm (∀ λX.(X ⇒ X) ⇒ X)

Lam λX.Lam λY.lam λx.lam λy.x                    : Tm (∀ λX.∀ λY.X ⇒ Y ⇒ X)

nincs                                            : Tm (∀ λX.X)

Lam λX.Lam λY.Lam λZ.lam{X⇒Y}λf.lam{Y⇒Z}λg.lam{X}λx.g·(f·x)  : Tm(∀λX.∀λY.∀λZ.(X⇒Y)⇒(Y⇒Z)⇒X⇒Z)
```

4 További típusok megadása

A `Three` típust megadjuk az alábbi módon.

```
Three := ∀ λX.X ⇒ X ⇒ X ⇒ X
```

8. (4 pont) Add meg a `one`, `two` operátorokat úgy, hogy teljesüljenek a β szabályok, és bizonyítsd be a $\text{Three}\beta_3$ szabályt egyenlőségi érveléssel! A `zero` operátort és az iterátort megadtuk.

```
zero : Tm Three

zero := Lam λX.lam λx.lam λy.lam λz.x

one : Tm Three

one := Lam λX. lam λx. lam λy. lam λz. y

two : Tm Three

two := Lam λX. lam λx. lam λy. lam λz. z

iteThree : Tm Three → Tm A → Tm A → Tm A → Tm A

iteThree t u v w := t · A · u · v · w

Threeβ₁ : iteThree zero u v w
                                                    =(definíció)
          (Lam λX.lam λx.lam λy.lam λz.x) · A · u · v · w
                                                    =(∀β)
          (lam λx.lam λy.lam λz.x) · u · v · w
                                                    =(⇒β)
          (lam λy.lam λz.u) · v · w
                                                    =(⇒β)
          (lam λz.u) · w
                                                    =(⇒β)
          u

Threeβ₂ : iteThree one u v w = v

Threeβ₃ : iteThree two u v w = (definíció)
          (Lam λX. lam λx. lam λy. lam λz. z) • A · u · v · w = (∀β)
          (lam λx. lam λy. lam λz. z) · u · v · w = (⇒β)
          (lam λy. lam λz. z) · v · w = (⇒β)
          (lam λz. z) · w = (⇒β)
          w
```

A szorzat típusokat az alábbi módon kódoljuk:

```
_×_ : Ty → Ty → Ty
A × B := ∀ λC.(A ⇒ B ⇒ C) ⇒ C
```

9. (5 pont) Add meg a szorzat típus konstruktorát, destruktorait és bizonyítsd be az egyik β szabályt! (A másiknak is teljesülnie kell, de nem kell levezetni.)

```
_,_ : Tm A → Tm B → Tm (A × B)


a , b := Lam λX. lam λf. f · a · b
```

```
fst : Tm (A × B) → Tm A

fst w := w • A · (lam λa. lam λb. a)

snd : Tm (A × B) → Tm B

snd w := w • B · (lam λa. lam λb. b)

×β₁ : fst (a , b) = (definíció)

          (a, b) • A · (lam λa. lam λb. a) = (definíció)

          (Lam λX. lam λf. f · a · b) • A · (lam λa . lam λb. a) = (∀β)

          (lam λf. f · a · b) · (lam λa. lam λb. a) = (⇒β)

          (lam λa. lam λb. a) · a · b = (⇒β)

          (lam λb. a) · b = (⇒β)

          a

×β₂ : snd (a , b) = b
```

A `Three` és a szorzat segítségével elkódoljuk a homogén ternáris összeg típust az alábbi módon:

```
3*_ : Ty → Ty
3*_ := Three × A
```

10. (6 pont) Add meg a konstruktorokat és a destruktort, és bizonyítsd az egyik β szabályt! (A többinek is teljesülnie kell.)

```
inj₁ : Tm A → Tm (3* A)

inj₁ u := (zero, u)

inj₂ : Tm A → Tm (3* A)

inj₂ u := (one, u)

inj₃ : Tm A → Tm (3* A)

inj₃ u := (two, u)

case : Tm (3* A) → (Tm A → Tm C) → (Tm A → Tm C) → (Tm A → Tm C) → Tm C

case w u₁ u₂ u₃ := iteThree (fst w) (u₁ (snd w)) (u₂ (snd w)) (u₃ (snd w))

3*β₁ : case (inj₁ w) u₁ u₂ u₃ =

iteThree (fst(zero, w)) (u₁ (snd(zero, w))) (u₂ (snd(zero, w))) (u₃ (snd(zero, w)))=(xβ₁,₂)

iteThree zero (u₁ w) (u₂ w) (u₃ w) = (Threeβ₁)

       u₁ w
```

$3*\beta_2$: case (inj_2 w) u_1 u_2 u_3 = u_2 w

$3*\beta_3$: case (inj_3 w) u_1 u_2 u_3 = u_3 w

11. (5 pont) Add meg, hogy az alábbi típusoknak hány elemük van! Az elsőt megadtuk példaként.

∀ λX.X ⇒ X	1
∀ λX.X ⇒ X ⇒ X	**2**
∀ λX.(X ⇒ X) ⇒ (X ⇒ X)	**∞**
∀ λX.(X ⇒ X ⇒ X) ⇒ (X ⇒ X)	**∞**
∀ λX.(X ⇒ X) ⇒ X	0
∀ λX.X	0
∀ λX.X ⇒ X ⇒ X ⇒ X ⇒ X	4
∀ λX.∀ λY.X⇒Y	0

Elkódoljuk a létezik típusoperátort, a konstruktorát, destruktorát és a β szabályát az alábbi módon.

```
∃ : (Ty → Ty) → Ty ∃ F := ∀ λC.(∀
λX.F X ⇒ C) ⇒ C mk∃ : (X : Ty) →
Tm (F X) → Tm (∃ F)
mk∃ X w := Lam λC.lam λu.u·X·w
un∃ : ((X : Ty) → Tm (F X) → Tm C) → Tm (∃ F) → Tm C
un∃ f u := u·C·(Lam λX.lam λw.f X w)
∃β : un∃ f (mk∃ X w)
                                                    =(def)
      (Lam λC.lam λu.u·X·w)·C·(Lam λX.lam λw.f X w)
                                                    =(∀β)
      (lam λu.u·X·w)·(Lam λX.lam λw.f X w)
                                                    =(⇒β)
      (Lam λX.lam λw.f X w)·X·w
                                                    =(∀β)
      (lam λw.f X w)·w
                                                    =(⇒β)
      f X w
```

12. (8 pont) Kódold el a végtelen mély bináris fák típusát, melyek a csomópontoknál A-t tartalmaznak! Add meg a három destruktort (amely kiveszi az A-t, megadja a fa bal illetve jobb oldali gyerekét), a generátorát, és bizonyítsd be a $Tree\beta_1$ és $Tree\beta_3$ szabályokat ($Tree\beta_2$ is teljesüljön)!

```
Tree : Ty → Ty


Tree A := ∃λC.((C ⇒ A) x ((C ⇒ C) x (C ⇒ C))) x C

item : Tm (Tree A) → Tm A


item w := _____________________________________

left : Tm (Tree A) → Tm (Tree A)


left w := _____________________________________

right : Tm (Tree A) → Tm (Tree A)
```

```
right w := ____________________________________
gen   : (Tm C → Tm A) → (Tm C → Tm C) → (Tm C → Tm C) → Tm C → Tm (Tree A)

gen {C} i l r c := ____________________________________________________________

Treeβ₁ : item (gen i l r c)

        i c

Treeβ₂ : left (gen i l r c) = gen i l r (l c)

Treeβ₃ : right (gen i l r c) =

        gen i l r (r c)
```

13. (5 pont) Add meg, hogy melyik típushoz melyik definíció tartozik! Kettőt megadtunk példaként. Az utolsónál használhatod a természetes számokat, rövidítsd `Nat`-nak!

természetes számok (Nat)	∀ λC.(C⇒C)⇒C⇒C
véges mélységű bináris fák extra információ nélkül	**∀ λC.(C ⇒ C ⇒ C) ⇒ C ⇒ C**
véges mélységű bináris fák, levélnél Three egy eleme	**∀ λC.(C ⇒ C ⇒ C) ⇒ (Three ⇒ C) ⇒ C**
kételemű típus	**∀ λX.X ⇒ X ⇒ X**
háromfelé ágazó végtelen mélységű fák, csomópontoknál A	∃ λC.(C ⇒ C × C × C) × (C ⇒ A) × C
A-kat tartalmazó végtelen hosszú lista	∃ λC.(C ⇒ C) × (C ⇒ A) × C
Végtelenfelé ágazó végtelen mélységű fák, csomópontoknál Three egy eleme	**∃ λC.(C ⇒ Nat ⇒ C) × (C ⇒ Three) × C**

5 Programozás

Most a korábban megadott típusokat használjuk programozásra. Az alábbiakban használd a `Tree` és `Three` típusok konstruktorait és destruktorait közvetlenül, nem kell a Church-kódolást kifejteni (akkor is megoldhatók ezek a feladatok, ha nem sikerült megoldani a 8. és a 12. feladatokat).

14. (3 pont) Add meg a háromelemű típusra a modulo 3 rákövetkezőt! Tehát egy olyan `suc : Tm Three → Tm Three` függvényt, melyre `suc zero = one`, `suc one = two`, `suc two = zero`.

    `suc t = iteThree t one two zero = t · Three · one · two · zero`

15. (2 pont) Add meg azt a végtelen mélységű bináris fát, amely mindenhol `one`-t tartalmaz (`Tm (Tree Three)` egy eleme)!

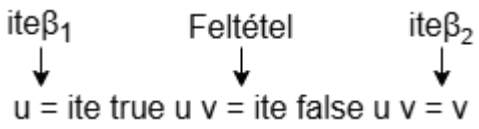
    `gen {C} (λc. one) (λc. c) (λc. c) c` ← bármilyen c-re jó, pl. `c := zero`

    `∃C ((lam (λc. one), (lam λc. c, lam λc. c)), c)`

16. (3 pont) Add meg azt a végtelen mélységű bináris fát, amelynek gyökerénél `zero` van, következő szintjén mindenhol `one` van, következő szintjén mindenhol `two`, következő szintjén mindenhol `zero`, következő szintjén mindenhol `one`, és így tovább!

    `gen (λc. c) suc suc zero`

17. (4 pont) Mutasd meg, hogy ha `zero = one`, akkor minden `u, v : Tm A`-ra `u = v`!

18. (3 pont) Igaz-e, hogy minden típusnak van eleme a nyelvünkben? Indoklást is kérünk.

    Nem, a System F-ben, amiről szó van, léteznek üres típusok, pl.: `∀ λX.X`

19. (3 pont) Belátható-e, hogy minden `t : Tm (A ⇒ A)`-ra `lam (λ x . t · x) = t`? Ha igen, vezesd le! Ha nem, indokold!

    Igen, ez lényegében a System F (amiről szó van) ⇒η szabálya.